WIRE LION ENGINEERING CLUB

HOLLOW HIVE

Distributed Raspberry Pi and ESP32 Animatronic Robot

FIRST PLACE | 2026 HALLOWEEN COMPETITION | UCCS



Hollow Hive animatronic pumpkin — cover image edited to remove people.

Technical Project Summary | University of Colorado Colorado Springs

PROJECT BACKGROUND	A student-built animatronic pumpkin designed to notice and react to visitors, later developed into a standalone robot.

Team: Kaeden Wellman and Jonathan B. | Technical report based on testing dated October 29, 2026

1. Project Overview

Hollow Hive began as a reactive animatronic pumpkin for the Wire Lion Engineering Club competition at the University of Colorado Colorado Springs. After the event, the team reused the electronics in a standalone robot. The design separates display and sensing work on a Raspberry Pi from time-sensitive actuator control on an ESP32.

The October 29, 2026 test confirmed both animated displays, automatic startup, the Pi-to-ESP32 command path, and leg positioning. It also documented unfinished integration work: distance messages did not reach the ESP32, face tracking was not yet demonstrated on the Pi, and the lift encoder failed to report the expected movement.

The system uses a distributed control architecture: the Raspberry Pi Zero 2 W creates the eye animation, reads the HC-SR04 sensor, and supports a proposed camera tracking workflow; the ESP32 DevKit V1 interprets UART commands and updates the servos, lift motor, and head-spin motor. The boards share a ground and use a one-way command channel in the tested configuration.

WHAT WAS ACTUALLY VERIFIED

The two round eye displays ran at 12.5 frames/s, starting about 40 seconds after power-up. Commands sent from the Pi were recognized by the ESP32, and the legs responded after firmware calibration. Other intended behaviors remained incomplete at the last recorded test.

2. Objective

The team, Hollow Hive, set out to build a pumpkin that notices people and reacts to them. After the competition, the pumpkin shell was retired and the electronics were reused as a standalone robot.

The project was divided into six tasks: animated eyes on round displays; detecting people; the Pi-to-ESP32 command link; leg and sway motion; lift and head-spin motor control; and power distribution. The intent was to build a responsive system that could display animated eyes, sense visitors, and command motion without requiring a laptop during normal startup.

3. Components and Materials

The following inventory organizes the hardware named in the submitted Hollow Hive project report. It describes the design and test configuration; it is not a verified purchasing list or a guarantee that every planned part was integrated.

Subsystem	Hardware / interface
Compute and displays	Raspberry Pi Zero 2 W; two 1.28-inch GC9A01 round SPI displays, 240 × 240 pixels
Sensing	HC-SR04 ultrasonic sensor; Raspberry Pi Camera Module 2 selected for face tracking
Motion control	ESP32 DevKit V1; two MG995 servos; two N20 encoder-equipped worm-gear motors
Motor driver	TB6612FNG dual channel; motor supply configured to 12.0 V

Subsystem	Hardware / interface
Power	2S LiPo (7.4 V, 2200 mAh); 12 V boost; 6 V buck; 10 A fuse; 5 V adapter for Pi

The two round displays and the Pi-to-ESP32 command path were demonstrated. Camera tracking was tested with simulated displays on a laptop, and the proposed Wi-Fi CSI sensor was not installed in the recorded build. The 10 kΩ lift-encoder pull-ups were identified as a possible fix, not as verified installed components.

4. Circuit Wiring Overview

Figure 1 follows the original team diagram: solid arrows carry commands and data, dotted arrows represent encoder feedback, and the dashed Wi-Fi CSI block is planned rather than demonstrated.

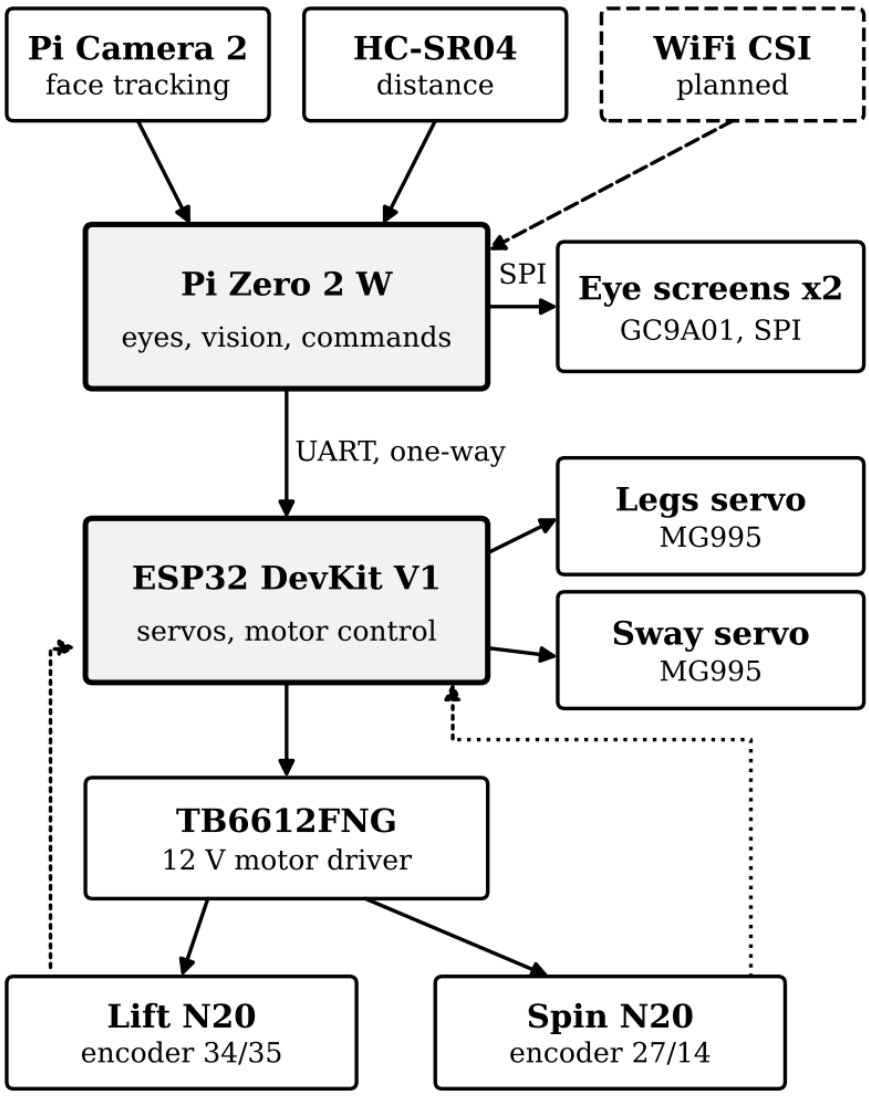


Figure 1. Original team system block diagram. Dashed Wi-Fi CSI block indicates planned work; dotted lines indicate encoder feedback paths.

Original control and sensor pin map

Connection	GPIO / signal
Pi → ESP32 UART	Pi GPIO14 / physical pin 8 → ESP32 GPIO16 (RX2); shared ground
Servos	ESP32 GPIO18 (legs), GPIO19 (sway)
Lift motor driver / encoder	Driver GPIO32 + 33; encoder GPIO34 + 35 (external pull-ups required)
Spin motor driver / encoder	Driver GPIO25 + 26; encoder GPIO27 + 14
Ultrasonic on Pi	Trigger GPIO23; Echo GPIO24 through 1 kΩ / 2 kΩ divider

The Pi and ESP32 must share ground. The HC-SR04 Echo line is a 5 V signal and was reduced with a 1 kΩ / 2 kΩ voltage divider before entering the Pi GPIO. The diagram is a functional architecture, not a fully verified electrical schematic showing every connector, protection device, and net.

5. Theory of Operation

Animated eyes on round displays

Two GC9A01 displays share SPI data, clock, data/command, and reset signals; each has a separate chip-select (CE0 and CE1). A Python graphics loop draws an eyeball with Pillow, transforms it with NumPy into RGB565 format, and sends matching frames to both screens. The idle motion changes gaze position every 0.4–1.8 seconds, recenters approximately 30% of the time, and blinks every 2–6 seconds. A system service launches the eye program when the Pi boots.

Display test	Recorded result
Frame rate	12.5 frames/s on both screens
Startup	Automatic, approximately 40 s after power-on
Power condition	Wi-Fi instability seen on laptop USB; resolved with the 5 V, 2.5 A wall adapter

Detecting and following visitors

The HC-SR04 sensor measures distance at the Pi. Its 5 V Echo output is reduced with a 1 kΩ / 2 kΩ divider before entering the 3.3 V GPIO input. The ultrasonic Python script can optionally send whole-centimeter values as “DIST n” UART messages, with -1 representing no valid target. A range below 100 cm is configured to initiate the automatic wake behavior; inactivity for 10 seconds is configured to return the robot to rest.

A Camera Module 2 and OpenCV Haar-cascade face detector were chosen for directional tracking on 320 × 240 grayscale frames. The team tested the camera logic on a laptop with simulated displays, but did not establish that it worked on the Pi Zero 2 W by the last report date. The earlier OV7670 + Arduino Uno route was dropped because memory was insufficient for the intended frame buffer.

Pi-to-ESP32 command link

The Pi’s GPIO14 transmit line (physical header pin 8) connects to ESP32 GPIO16 (RX2). Both processors share ground. The UART link operates at 115200 baud with newline-terminated text commands. The ESP32-to-Pi return link could not be validated, so the recorded build uses a one-way Pi-to-ESP32

channel. An initial noise issue was reduced by sending a blank line at connection and ignoring nonprintable characters.

Command	Intended function
STAND / SIT	Leg positions: 7° extended / 133° tucked
SWAY 0–3	Sway amplitude from off to highest setting
LIFT 0–100	Lift target as percent of nominal travel
SPIN -180…180	Requested head angle relative to forward
DIST n; AUTO 0/1	Distance input and automatic wake control
MOTOR L/S n; HOME	Half-second motor test; lift homing
STATUS / STOP	Status output; stop commanded motion

Leg and sway motion

Two MG995 servos manage leg extension and side-to-side sway. The ESP32 outputs 50 Hz control pulses using 14-bit LEDC resolution. The leg target transitions gradually at 90°/s rather than jumping to its final angle. Firmware revisions corrected the direction and limited the travel to a standing setpoint of 7° and a seated setpoint of 133°. Sway is defined as a sinusoidal motion with a gradually adjusted amplitude; its final-session behavior was not recorded.

Lift and head-spin motion

Two 12 V N20 worm-gear motors use the dual-channel TB6612FNG driver: one raises the top assembly and one rotates the head. Encoder interrupts track movement, and a 10 ms loop moves a target setpoint toward the requested position. Proportional control applies a minimum drive level of 60/255 when movement is required. Firmware is intended to stop a motor after 800 ms without encoder progress, and the head command is limited to ±180° to reduce cable twisting.

The control constants are preliminary rather than measured calibration values: the appendix uses 3,000 encoder counts for full lift travel and 2,100 counts for a full head revolution. The team reported only two lift encoder counts during a powered test, so closed-loop lift positioning was not confirmed. ESP32 GPIO34/35 lack internal pull-ups; the proposed next step was to add 10 kΩ external pull-ups to 3.3 V and retest.

Power distribution

A 2S LiPo powers the motion subsystem through a 10 A fast-blow fuse and an on/off switch. A boost converter was set to 12.0 V for the motor driver; a buck converter was set to 6.0 V for the MG995 servos. The Pi was supplied by a separate 5 V, 2.5 A wall adapter. The ESP32 used a laptop USB connection during development and the Pi 5 V pin in the intended final configuration, but not both at the same time. Common ground reference is required for both boards and actuator supplies.

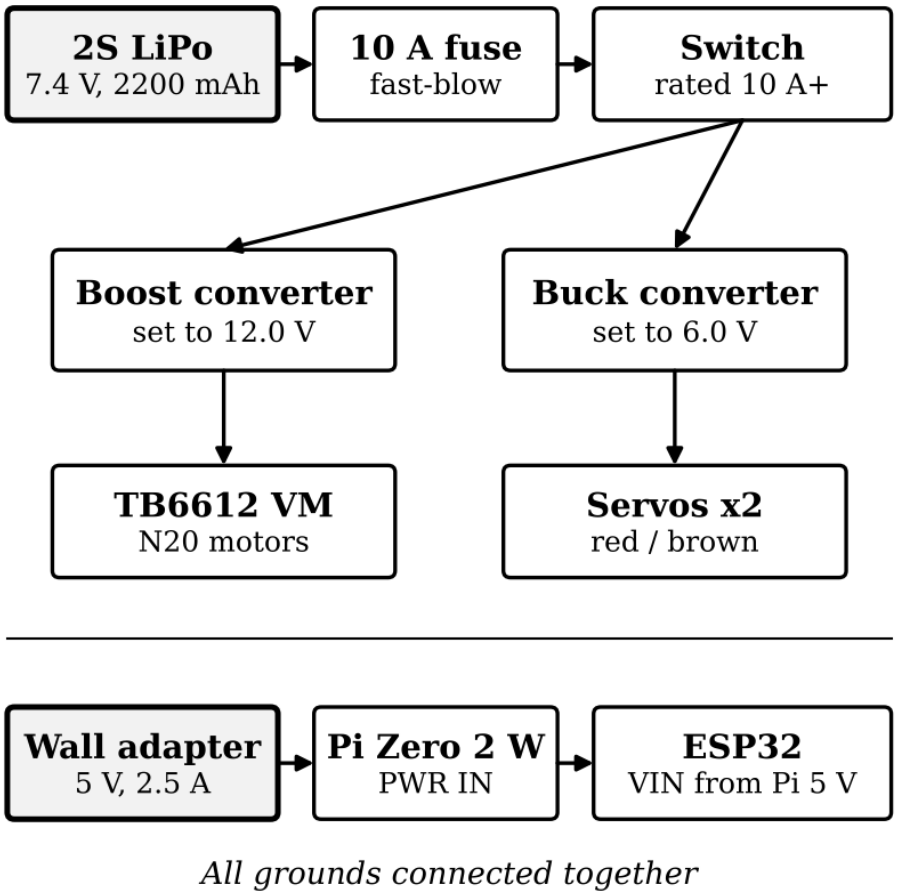


Figure 2. Original team power distribution diagram. Converter output settings were checked by multimeter before connecting loads.

6. Assembly Procedure

The stages below follow the previously prepared Hollow Hive replication and test checklist. They are instructions for a future builder, not a claim that every stage was successfully completed during the competition.

Stage	Expected verification / caution
1. Assemble power	With motors disconnected, measure 12.0 V and 6.0 V rails, verify polarity, and verify fuse and wiring ratings for the actual build.
2. Start Raspberry Pi	Verify both round displays initialize and animation runs after boot; confirm power supply does not cause undervoltage or Wi-Fi dropouts.
3. Establish UART	Share ground; connect Pi TX (GPIO14, pin 8) to ESP32 RX2 (GPIO16); send STAND, SIT and STATUS from a controlled test setup.
4. Confirm servos	Test STAND and SIT with mechanical travel clear; verify setpoints 7° and 133° on the assembled mechanism.
5. Test encoders	Check ESP32 encoder pin assignments and add required external pull-ups for lift encoder inputs 34/35; measure travel counts.

Stage	Expected verification / caution
6. Test distance	Run ultrasonic.py with the --arduino /dev/serial0 option; confirm that DIST values arrive at the ESP32.
7. Validate camera and sway	Only then validate Camera Module 2/OpenCV tracking on Pi, head spin, automatic wake, and continuous sway.

SOURCE CODE AND REPRODUCIBILITY

The accompanying source archive contains the three code listings present in the original appendix: pumpkin_esp32_v4.ino, ultrasonic.py, and link_test.py. The authors identified round_eyes.py and emotions.py as still stored on the Pi, so the submitted materials are not yet a complete build-from-scratch release. The extracted scripts are provided as submitted, not independently tested.

7. Testing and Troubleshooting

The authors tested subsystems individually and logged the final-session status on October 29, 2026. The table below preserves the difference between demonstrated functions and planned or incomplete functions.

Task	Evidence recorded	Status
01 Animated eyes	Both displays at 12.5 fps; auto-start at boot	Verified
02 Visitor detection	HC-SR04 read on Pi; DIST messages not seen at ESP32; camera test only on laptop	Partial
03 Serial commands	Pi → ESP32 commands recognized; return path did not work	Partial
04 Leg / sway	Leg STAND and SIT worked after two firmware revisions; sway not logged	Partial
05 Lift / head spin	Firmware responded; lift counted 2 pulses; spin motor not tested	Incomplete
06 Power subsystem	12.0 V and 6.0 V verified on converters; 10 A fuse installed	Verified setup

Troubleshooting recorded by the team

Power and networking. The Pi repeatedly lost Wi-Fi while powered by a laptop USB port; the 5 V, 2.5 A wall adapter resolved the dropout, and the Pi power check reported 0x0.

Distance transmission. The ultrasonic sensor reported distance on the Pi, but its readings did not reach the ESP32. The team identified the send option, UART wiring, and port ownership as possible causes; the exact failure was not established.

Serial return path. Commands from the Pi reached the ESP32, while ESP32 replies did not reach the Pi. The design therefore used one-way commands; a blank first line and filtering of nonprintable characters addressed an initial noise problem.

Servos and encoders. The STAND and SIT directions required two firmware revisions. The lift encoder returned only two counts during a high-power test; GPIO34 and GPIO35 have no internal pull-ups. The sway and spin motors were not recorded as successfully tested at the last session.

Camera integration. Face tracking and recording were tested on a laptop with simulated displays, not on the Pi Zero 2 W in the final build.

The table and troubleshooting notes record the team's October 29, 2026 results and should not be interpreted as independent verification or as evidence of complete autonomous operation.

8. Engineering Concepts Learned

- **Power first.** Insufficient USB power caused intermittent Wi-Fi behavior that initially appeared to be a networking fault.
- **GPIO capability matters.** Pins GPIO34 and GPIO35 are input-only and do not provide internal pull-up resistors; an encoder connection cannot be validated by a firmware setting alone.
- **Prove links separately.** A basic UART test narrowed the problem to sensor-message forwarding instead of assuming all serial traffic was broken.
- **Simplify before integration.** The initial gimbal motor concept was replaced with N20 motors, reducing control complexity.
- **Keep wiring notes current.** The team revised hardware during the build and identified outdated wiring notes as a risk.

9. Possible Improvements

The team's report identifies the following next steps. These items remain future work unless additional testing documents them.

- Add and measure external lift-encoder pull-ups; retest the encoder response under motion.
- Trace the ultrasonic send option, UART port ownership and connection until DIST commands reach the ESP32.
- Measure real encoder counts for full lift travel and full head rotation before relying on positional commands.
- Demonstrate head-spin, sway and face tracking on the intended hardware, with repeatable test records.
- Assess the proposed Wi-Fi CSI sensing concept using a second ESP32; it was not part of the verified build.
- Copy the current round_eyes.py and emotions.py versions into the project archive and confirm all authors approve public source release.

10. Completed Project and Contributors

Hollow Hive was entered in the Wire Lion Engineering Club's 2026 Halloween competition and earned First Place as selected by the club competition organizer. After the competition, the team reused the project electronics as the basis for a standalone animatronic robot; subsystem integration was still ongoing at the last documented test.

Role / recognition	Contributor / details
Project team	Kaeden Wellman; Jonathan B.

Role / recognition	Contributor / details
Original technical report author	Kaeden Wellman
Original report team member	Jonathan B.
Competition result	First Place — Wire Lion Engineering Club 2026 Halloween competition
Institution	University of Colorado Colorado Springs (UCCS)

Original report author: Kaeden Wellman. Team member: Jonathan B. Both are credited as the Hollow Hive project team in the competition report. Jonathan B. is a mechanical engineering student with an interest in mechatronics, robotics, and mechanical design. He has hands-on experience with 3D printing, CAD modeling, electronics, and microcontroller programming. His current projects focus on combining mechanical systems, sensors, and embedded controls to develop interactive robotic systems. Kaeden Wellman is a first-year Electrical Engineering student and Division II track athlete at the University of Colorado Colorado Springs. On Hollow Hive, he led system integration across the Raspberry Pi and ESP32, from the serial link and power wiring to hardware testing. He aims to work where electrical and software engineering meet, with a growing interest in AI and machine learning, from computer vision to WiFi sensing.

Project recognition: First Place, Wire Lion Engineering Club Halloween competition, October 2026 (as stated by the club's competition organizer). This public-facing formatting is editorial; it does not change the technical results recorded by the project team.

Demonstration video: https://www.youtube.com/watch?v=BGYTVnDWbCk&t=22s

References and original material

[1] Hollow Hive: A Distributed Raspberry Pi and ESP32 Animatronic Robot, original lab report by Kaeden Wellman and Jonathan B., dated test results October 29, 2026.

[2] OpenCV, pretrained Haar cascade: haarcascade_frontalface_alt.xml. https://github.com/opencv/opencv/tree/master/data/haarcascades

[3] Espressif Systems, esp-radar, v0.3.4. https://components.espressif.com/components/espressif/esp-radar/versions/0.3.4/readme

[4] Espressif Systems, esp_wifi_sensing, v0.1.1~3. https://components.espressif.com/components/espressif/esp_wifi_sensing/versions/0.1.1~3/readme

[5] P. Viola and M. Jones, "Rapid object detection using a boosted cascade of simple features," Proc. IEEE CVPR, 2001, pp. I-511–I-518.

[6] G. Bradski, "The OpenCV Library," Dr. Dobb's Journal of Software Tools, 2000.

[7] Adafruit Industries, Adafruit_Monster_Eyes source repository. https://github.com/adafruit/Adafruit_Monster_Eyes